

Sistema de Gestión VetSalud

Arquitectura de Persistencia Políglota con Motores NoSQL

Grupo 10 | Base de Datos II | ITBA

- 61105 Causse Juan Ignacio
- 64332 Liu Javier Emmanuel
- 64292 Rivas Nicolás

Contexto

- VetSalud S.A. requiere modernizar su backoffice, migrando de planillas de cálculo a una solución de software escalable.
- El sistema debe gestionar eficazmente entidades dispares como pacientes, veterinarios, historiales y consultas médicas.
- Existe una necesidad crítica de mantener el control de inventario y stock farmacéutico actualizado en tiempo real.
- La solución requiere una arquitectura políglota para asignar el motor de base de datos ideal a cada caso de uso.

Estrategia de Persistencia Políglota



MongoDB (Datos Clínicos)

Elegida como base de datos principal por su versatilidad documental. Su capacidad para ejecutar agregaciones y ensambles (lookups) permite gestionar historiales clínicos complejos de forma normalizada, evitando el desborde del tamaño máximo por documento.



Apache Cassandra (Stock)

Seleccionada para el control del inventario farmacéutico. Frente a una altísima frecuencia de operaciones de lectura y escritura, Cassandra garantiza rendimiento distribuido. El ID de producto actúa como partition key para balancear la carga uniformemente.

Inicialización y Optimización de Datos



Inicialización Automatizada: Carga de 6 datasets CSV iniciales mediante scripts de configuración montados nativamente en los volúmenes de ambas bases de datos.



Índices SAI en Cassandra: Implementación de Storage Attached Indexes para permitir búsquedas analíticas eficientes por desigualdades (ej. stock menor a 50 unidades).



Vistas Agregadas: Creación de pipelines complejos en MongoDB (vw_ingresos_veterinario) para simplificar la generación de reportes financieros a demanda.



Validación Estricta: Control de tipos de datos a nivel API y validación de concurrencia para evitar discrepancias durante actualizaciones atómicas de stock.

Arquitectura de Aplicación

express

NGINX



Backend API

Desarrollada en Express.js (Node.js). Conexión nativa a MongoDB mediante Mongoose y a Cassandra vía su driver oficial, centralizando la lógica de negocio.



Frontend SPA

Single Page Application veloz construida en Vanilla JS. Servida eficientemente por NGINX, consume la API dinámicamente y expone los datos al usuario.



Documentación

Generación automática de OpenAPI/Swagger UI. Disponible en un endpoint dedicado para probar y documentar todas las consultas desarrolladas en tiempo real.

Infraestructura



	Name	Image	Port(s)
🔵	bd2-tpo	-	-
●	g10-cassandra	cassandra:latest	9042:9042 ↗
●	g10-mongo	mongo:latest	27017:27017 ↗
○	g10-cassandra-init	bd2-tpo-cassandra-init	
●	g10-api	bd2-tpo-api	3000:3000 ↗
●	g10-frontend	nginx:alpine	8080:80 ↗

El proyecto se despliega íntegramente mediante Docker Compose, aislando los servicios en una red interna y asegurando un entorno reproducible.

Se integran contenedores dedicados para la API, el Frontend, MongoDB y Cassandra. Destaca el uso de un sidecar efímero para la inicialización segura en Cassandra.

```
docker-compose.yml
  ▶ Run All Services
1  services:
2
3    # MongoDB Container
4    ▶ Run Service
5    mongo: ...
23
24    # Apache Cassandra Container
25    ▶ Run Service
26    cassandra: ...
38
39    # Apache Cassandra Init Container
40    ▶ Run Service
41    cassandra-init: ...
56
57    # Express.js API Container
58    ▶ Run Service
59    api: ...
78
79    # NGINX Frontend Container
80    ▶ Run Service
81    frontend: ...
90
91  volumes:
92  >  mongo_data: ...
95  >  cassandra_data: ...
98
```

Frontend

GENERAL

 Inicio

CONSULTAS MONGODB

-  Query 1: Pacientes activos
-  Query 2: Consultas en seguimiento
-  Query 3: Historial de paciente
-  Query 4: Propietarios con +1 paciente
-  Query 5: Veterinarios últimos 60 días
-  Query 6: Vacunas vencidas
-  Query 7: Top 5 diagnósticos

CONSULTAS CASSANDRA

 Query 8: Stock bajo (<50 u.)

CONSULTAS MONGODB

-  Query 9: Controles < \$5.000
-  Query 10: Pacientes por sucursal
-  Query 11: Ingresos por veterinario
-  Query 12: Propietarios sin consultas

OPERACIONES DE ESCRITURA

-  Query 13: ABM Propietarios
-  Query 14: Nueva consulta médica
-  Query 15: Actualizar stock

 Query 8 — Stock bajo (<50 u.)

Productos farmacéuticos con menos de 50 unidades en stock y su proveedor

Resultados

☒ Mostrar Identificadores

Cassandra

ID PRODUCTO	PRODUCTO	CATEGORIA	UNIDADES	PRECIO UNITARIO	VENCIMIENTO	PROVEEDOR
PRD007	Cefalexina 500mg	Antibiótico	35	1100	2027-02-15	VetFarma SA
PRD004	Frontline Spray	Antiparasitario	32	3500	2027-01-10	MediAnimal
PRD035	Nebulizador Veterinario	Equipamiento	5	18000	2030-01-01	VetEquip SA
PRD031	Carprofeno 50mg	Antiinflamatorio	11	2400	2027-09-01	MediAnimal
PRD030	Metronidazol 250mg	Antibiótico	29	1300	2027-03-25	VetFarma SA
PRD014	Clorhexidina Spray	Antiséptico	38	1900	2026-10-15	MediAnimal
PRD017	Lidocaína 2%	Anestésico	12	3400	2026-12-10	MediAnimal
PRD018	Atropina 1mg	Medicamento	30	2100	2027-04-22	BioVet SRL
PRD023	Doxiciclina 100mg	Antibiótico	17	1450	2027-07-15	VetFarma SA
PRD020	Collar Antipulgas	Antiparasitario	20	4200	2027-06-10	PetCare SA
PRD032	Desparasitante Oral	Antiparasitario	47	1250	2027-04-10	PetCare SA
PRD036	Oxitetraciclina 100mg	Antibiótico	33	1550	2027-08-12	VetFarma SA
PRD010	Pipeta Canina M	Antiparasitario	18	2900	2026-08-05	PetCare SA
PRD034	Termómetro Digital	Equipamiento	8	6500	2030-01-01	VetEquip SA
PRD028	Gotas Óticas	Otológico	41	1700	2026-12-31	PetCare SA
PRD006	Ketamina 10%	Anestésico	21	5800	2026-05-01	MediAnimal
PRD009	Enrofloxacin 50mg	Antibiótico	28	1350	2027-01-10	MediAnimal

Readme

Documentos Importantes

En el directorio `docs` se encuentran los siguientes documentos:

- `Enunciado.pdf` - Enunciado del trabajo práctico.
- `Informe.pdf` - Informe de entrega.
- `docker-compose.md` - Documentación de la arquitectura en Docker Compose.

Rutas

A continuación se detallan los servicios disponibles, sus rutas y puertos.

Servicio	Ruta	Puerto
Frontend	<code>/</code>	8080
Documentación Swagger	<code>/api-docs</code>	3000

Construcción y Ejecución

Para construir y ejecutar el proyecto es necesario contar con [Docker Engine](#) y [Docker Compose](#) instalados.

Una vez instalados, se puede construir y ejecutar el proyecto ejecutando el siguiente comando en la raíz del proyecto:

```
docker compose up -d --build
```


Demostración

¡Muchas gracias!

Grupo 10 | Base de Datos II | ITBA